



Security Assessment

Final Report



AAVE Risk Steward

July 2026

Prepared for AAVE Labs

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
Findings Summary.....	9
Severity Matrix.....	9
Detailed Findings.....	10
Low Severity Issues.....	11
L-01: Risk council can reverse governance CF=0 decision and append new dynamic keys.....	11
L-02: Global cap bounds are not per hub.....	13
L-03: Active LST price cap can be lifted through a snapshot update.....	14
L-04: Same-value updates reset the debounce timestamp.....	16
Informational Severity Issues.....	17
I-01: Inconsistent error style in the constructor.....	17
Disclaimer.....	18
About Certora.....	18

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
AAVE-RiskSteward	https://github.com/ave-dao/aave-v4-risk-stewards	4347d13 (initial)	Solidity

Project Overview

This document describes the security review of **AAVE-RiskSteward**. The work was undertaken from **July 6th, 2026** to **July 13th, 2026**.

The following contract list is included in our scope:

src/interfaces/IRiskSteward.sol

src/RiskSteward.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The **RiskSteward** is a bounded parameter-update guardian placed in front of the Aave v4 Hub/Spoke configurators and the CAPO price-cap oracle adapters. Its purpose is to let a *Risk Council* push frequent, routine risk-parameter adjustments without a full governance vote, while a trusted *Owner* (governance) constrains how far and how fast each parameter may move. It is the v4 successor to the Aave v3 Risk Steward and reuses the same validation shape and, for the oracle paths, the same CAPO adapter contracts and setter signatures.

The contract exposes two privileged roles:

- **Owner** (`Ownable2Step`): sets the single global risk configuration via `setConfig` and manages the address restriction denylist via `setAddressRestricted`. The owner is the trust root – it chooses every per-parameter bound and can revoke the steward's downstream permissions and redeploy.
- **Risk Council** (`RISK_COUNCIL`, immutable): the only address permitted to invoke the nine update entrypoints. It operates strictly within the owner-configured bounds.

The steward governs the following risk-parameter families:

- **Hub-asset interest rate** per `(hub, asset)`: `optimalUsageRatio`, `baseDrawnRate`, `rateGrowthBeforeOptimal`, `rateGrowthAfterOptimal`.
- **Hub-spoke caps** per `(hub, spoke, asset)`: `addCap`, `drawCap`.
- **Spoke reserve config** per `(spoke, hub, asset)`: `collateralRisk`.
- **Spoke dynamic reserve config** per `(spoke, hub, asset)`: `collateralFactor`, `maxLiquidationBonus` — via both `updateDynamicReserveConfigs` (mutating an existing `dynamicConfigKey`) and `addDynamicReserveConfigs` (appending a new key), governed by two independent bound sets.
- **Spoke-global liquidation config** per `spoke`: `targetHealthFactor`, `healthFactorForMaxBonus`, `liquidationBonusFactor`.
- **CAPO oracle price caps** per `oracle`: the LST adapter's `maxYearlyRatioGrowthPercent/snapshotRatio/snapshotTimestamp`, the stable adapter's `priceCap`, and the Pendle adapter's `discountRatePerYear`.

Every numeric change passes a uniform validation model: a `KEEP_CURRENT` sentinel skips a field; a per-parameter debounce (`minDelay`) rate-limits successive changes; a range check bounds the change to the owner-configured `maxPercentChange` (relative BPS or absolute delta, per the field's `isChangeRelative` mode); a set of parameters (`collateralFactor`, `maxLiquidationBonus`, and the liquidation-config fields) may not be set to zero; every touched address (hub, spoke, asset, oracle) must be unrestricted; and every update must route through the owner-registered configurator. Out-of-scope fields must carry their `KEEP_CURRENT` sentinel, so the steward can only ever modify the whitelisted risk fields.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Scope and Trust Assumptions

The review focused on the correctness and security of `RiskSteward.sol` and its interface, including the per-parameter validation model (debounce, range, zero-guards), the restriction and configurator-matching controls, the sentinel handling, and the interaction with the downstream config engines, CAPO oracle adapters and the spoke and hub implementations.

The following are treated as trust assumptions – they were not verified in depth and must hold for the steward to operate safely:

- **The owner is fully trusted.** The owner is Aave governance (a timelocked executor / DAO). It deliberately chooses every `minDelay`, `maxPercentChange`, and `isChangeRelative` value, the configurator registrations, and the restriction denylist. Harm requiring owner misconfiguration, ownership renouncement, or deliberately loose bounds is out of scope.
- **Risk Council is semi-trusted and recoverable.** The council is expected to act in good faith within its granted powers, but is treated as potentially key-compromised or buggy. The intended incident response is for governance to revoke the steward's role on the Hub/Spoke side and redeploy with a new council; `RISK_COUNCIL` is immutable by design, with no in-contract rotation or pause. Consequently, the owner's monitoring and revocation latency is the effective backstop against a misbehaving council.
- **No untrusted actor can mutate state.** All nine update entrypoints are `onlyRiskCouncil`; both configuration setters are `onlyOwner`. The steward holds no funds and transfers no tokens.
- **CAPO adapters enforce their own bounds.** The LST, stable, and Pendle price-cap adapters correctly enforce their internal safety constraints – the LST adapter's snapshot monotonicity, `MINIMUM_SNAPSHOT_DELAY`, `MAXIMUM_SNAPSHOT_TERM`, and `isCapped()`; the stable adapter's `CapLowerThanActualPrice`; and the Pendle adapter's `MAX_DISCOUNT_RATE_PER_YEAR` and sub-100% discount checks. The steward deliberately delegates the bounding of oracle-level parameters to these adapters and mirrors the v3 Risk Steward's assumptions.

- **Correct deployment and permissioning.** The steward is granted the appropriate configurator-domain and oracle ACL admin roles, the configurator addresses in `Config` are correct, and all in-scope oracles are genuine CAPO adapters of the type matching the entrypoint used to update them.

Security Considerations and Threat Analysis

Bounded Parameter Authority. The steward exists to constrain the semi-trusted Risk Council to bounded, rate-limited parameter changes; two guarantees underpin every entrypoint. Threats include:

- Moving a parameter further than the owner intended in a single action – prevented by the range check, which caps each change at `maxPercentChange` of the current value. Because validation runs over the whole input array before any change is applied, packing multiple entries into one call cannot compound past the bound.
- Changing a parameter more frequently than the owner intended – prevented by the per-parameter debounce (`minDelay`), stamped on every applied change and enforced against the pre-transaction timestamp.
- Drifting a parameter far in a harmful direction over many windows – inherent to any rate-limiter: each step stays within `maxPercentChange`, and the owner's monitoring, restriction, and revoke-and-redeploy capability is the backstop.

Governance Authority. The steward must only touch its whitelisted risk fields and must not remove a safety property reserved for governance. Threats include:

- Modifying a field outside the steward's mandate – prevented by the `KEEP_CURRENT` enforcement, which forces every out-of-scope field to its sentinel and routes every update through the owner-registered configurator.
- Removing a safety property by zeroing a protected parameter – prevented by `InvalidUpdateToZero` on `collateralFactor`, `maxLiquidationBonus`, and the liquidation-config fields; a collateral tier that governance has disabled (`collateralFactor = 0`) cannot be re-enabled by the council, guarded on both the add and update paths. One finding is connected to this property.

Oracle Price Integrity. The price caps must prevent an anomalous or manipulated exchange rate from over-valuing collateral. Threats include:

- Under-valuing a stable to force liquidations – prevented by the stable adapter's `CapLowerThanActualPrice`, which rejects any cap below the current market price.
- Crashing the LST reported price by capping below market – prevented by the `!isCapped()` post-condition on the update.
- Mispricing a Pendle PT via the discount rate – bounded per step by the steward and by the adapter's `MAX_DISCOUNT_RATE_PER_YEAR` ceiling.

For the LST adapter the steward bounds the growth rate and delegates cap-level bounding – snapshot monotonicity and the 180-day term – to the adapter, as noted under Scope and Trust Assumptions.

Restriction and Scope Control. `setAddressRestricted` is the owner's lever to carve addresses out of the steward's reach. Threats include:

- An update touching an address the owner has excluded – rejected by `_requireNotRestricted` for every hub, spoke, asset, and oracle in the call.
- Routing an update through an unauthorized configurator – rejected by the configurator-match check against the owner's `Config`.

Restrictions are per-address with no cascade: to freeze a market's spoke-global liquidation configuration the owner restricts the spoke, since a hub restriction does not reach it.

Availability. The steward holds no funds and transfers no value, so it presents no fund-loss or asymmetric denial-of-service surface. Validation that passes but reverts at a downstream engine or adapter is a failed transaction with no state change.

In all categories, the steward's guarantees reduce to: no non-privileged actor can act, every council action is rate- and magnitude-bounded per step, and the owner retains monitoring, restriction, and revoke-and-redeploy as the ultimate backstop.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	4	4	-
Informational	1	1	-
Total	5	5	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	Risk council can reverse governance CF=0 decision and append new dynamic keys	Low	Acknowledged
L-02	Global cap bounds are not per hub	Low	Acknowledged
L-03	Active LST price cap can be lifted through a snapshot update	Low	Acknowledged
L-04	Same-value updates reset the debounce timestamp	Low	Acknowledged
I-01	Inconsistent error style in the constructor	Informational	Acknowledged

Low Severity Issues

L-01: Risk council can reverse governance CF=0 decision and append new dynamic keys

Severity: Low	Impact: Low	Likelihood: Low
Files: src/RiskSteward.sol – updateDynamicReserveConfigs	Status: Acknowledged	

Description:

When governance deliberately sets `collateralFactor = 0` on the latest dynamic config key for a reserve (by calling `Spoke.addDynamicReserveConfig` directly), it signals that the collateral tier is intentionally disabled.

Only governance should be able to reverse this decision.

In `validateAddDynamicReserveConfigs` we have `require(ref.collateralFactor != 0, NoExistingDynamicConfig());` that does not allow risk council to create a new dynamic key if the prev CF is 0.

This however can be bypassed with `updateDynamicReserveConfigs` so the Risk Council can: Update the CF=0 key to a small positive value → Append a new dynamic config key with non zero CF

Recommendations:

In `_validateDynamicReserveConfigs` (the update path), add a guard that blocks updates when the current value is zero: `require(current.collateralFactor != 0, CannotUpdateFromZero());`



This ensures that a $CF=0$ key set by governance remains protected and only governance can restore it to a non-zero value.

Customer Response:

Acknowledged.

L-02: Global cap bounds are not per hub

Severity: Low	Impact: Low	Likelihood: Low
Files: src/RiskSteward.sol	Status: Acknowledged	

Description:

`_config.hub` / `_config.spoke` is a single global config applied uniformly to every (`hub`, `spoke`, `asset`) update. The same `maxPercentChange` and `minDelay` govern changes across every hub the steward manages, so there is no way to set stricter bounds for one hub and looser bounds for another. Hubs with different risk profiles are forced to share one policy: bounds loose enough for one hub apply to all others.

Recommendations:

Support per-hub bounds, so `maxPercentChange` and `minDelay` can be configured independently for each hub.

Customer Response:

Acknowledged. We plan to enable per hub bounds as needed with a separate risk steward-hub configurator combination (i.e. separate class of hubs will be governed by an independent configurator) to reduce complexity.

L-03: Active LST price cap can be lifted through a snapshot update

Severity: Low	Impact: Low	Likelihood: Low
Files: src/RiskSteward.sol – updateLstPriceCaps()	Status: Acknowledged	

Description:

After applying new cap parameters, `_executeLstPriceCaps` requires `!isCapped()` so the update cannot immediately cap the oracle:

```
IPriceCapAdapter(oracle).setCapParameters(updates[i].priceCapUpdateParams);
require(!IPriceCapAdapter(oracle).isCapped(), InvalidPriceCapUpdate());
```

This check is one-directional: it blocks setting the cap *below* the live ratio, but not raising it. `_validateLstPriceCaps` rate-limits only `maxYearlyRatioGrowthPercent` (via `debounce + maxPercentChange`) and checks `snapshotRatio / snapshotTimestamp` only for non-zero and `snapshotRatio <= currentRatio`.

Nothing constrains how far a re-snapshot moves the effective `maxRatio`, so the Risk Council can release an already-active cap by re-snapshotting at the live ratio.

Example (wstETH/ETH):

- Live ratio `1.185`, snapshot 30 days ago at `1.180` with 10% yearly growth → `maxRatio ≈ 1.190`. Not capped, prices normally.
- Live ratio spikes to `1.250` (e.g. exchange-rate manipulation) → `isCapped() = true`, so the cap holds the reported price at `1.190`, protecting against over-borrowing.
- The council submits `snapshotRatio = 1.250` (passes `≤ currentRatio`), a fresh `snapshotTimestamp` (the adapter requires it to be strictly greater than the stored one and `≤ now - MINIMUM_SNAPSHOT_DELAY`), growth unchanged → `maxRatio ≈ 1.250` → `isCapped() =`

`false`, the check passes, and the oracle now reports `1.250`. Users can borrow against the inflated value → bad debt when the ratio corrects.

The impact is bounded. `latestAnswer()` reports `min(liveRatio, maxRatio)`, so the council can never push the reported price *above* the true (here manipulated) ratio – it can only remove the protective ceiling, not fabricate a higher price. A loss therefore requires the exchange rate itself to be manipulated, and the only reachable direction is releasing an already-active cap, which reduces to a trust assumption on the Risk Council.

Recommendations:

If the Risk Council is trusted enough to remove an active cap via a config update, leave the code as is. Otherwise, add a `require(!isCapped())` check in `_validateLstPriceCaps` so the steward cannot release an active cap.

Customer Response:

Acknowledged. Risk council is trusted enough to remove an active cap via a config update, same implementation exists on the v3 risk steward.

L-04: Same-value updates reset the debounce timestamp

Severity: Low	Impact: Low	Likelihood: Low
Files: src/RiskSteward.sol	Status: Acknowledged	

Description:

`_validateParamUpdate` skips a field entirely when it carries the `KEEP_CURRENT` sentinel, so a `KEEP_CURRENT` field does not update the debounce timestamp. However, providing the actual current value (instead of `KEEP_CURRENT`) does bump the debounce: the range check passes with `diff = 0`, and the execute functions stamp the timestamp because the input is not the sentinel.

The result is inconsistent with `KEEP_CURRENT`: two ways of expressing "no change" behave differently – one leaves the debounce untouched, the other resets it.

The only scenario where the change matters is when updates are automated and, for whatever reason, submit the same values while `minDelay` is in play. If a real update then has to be made, the full `minDelay` needs to elapse again since the last no-op update before the genuine change is allowed.

Recommendations:

For consistency with `KEEP_CURRENT`, no-op values should not consume the debounce and bump the timestamp.

Customer Response:

Acknowledged. Intentional for no-op values to consume debounce.

Informational Severity Issues

I-01: Inconsistent error style in the constructor

Files:

RiskSteward.sol

Description:

`require(riskCouncil_ != address(0))` (RiskSteward.sol:59) is a bare `require` with no custom error, while every other guard in the contract uses named errors.

Recommendations:

Add a named error (e.g., `InvalidRiskCouncil()`), or accept as is.

Customer Response:

Acknowledged.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.